

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэкенд-разработка

Отчёт

Домашнее задание №4
«Технический дизайн микросервисной
архитектуры»

Выполнил:
Клименков Владислав

Группа:
К3341

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задачи

Разделить монолитное бэкенд-приложение (платформа для обмена рецептами и кулинарный блог) на микросервисы, придерживаясь концепции database-per-service. Спроектировать микросервисную архитектуру: определить взаимосвязи между микросервисами, способы их взаимодействия, разделение базы данных, а также описать эндпоинты для межсервисного взаимодействия.

Ход работы

1. Анализ монолитного приложения

Исходное монолитное приложение ('legacy-monolith/') построено на стеке:

- Express.js — веб-фреймворк
- PostgreSQL — база данных
- Prisma ORM — работа с БД
- JWT — аутентификация

Сущности монолита:

- User — пользователи (id, username, password, firstName, lastName, about, role)
- Recipe — рецепты (id, authorId, title, description, difficulty, isPublished)
- RecipeMedia — медиафайлы рецепта
- RecipeStep — шаги приготовления
- RecipeStepMedia — медиафайлы шагов
- DishType — типы блюд
- RecipeDishType — связь рецептов с типами блюд
- Ingredient — ингредиенты
- RecipeIngredient — связь рецептов с ингредиентами
- Comment — комментарии
- CommentLike — лайки комментариев

- SavedRecipe — сохранённые рецепты
- RecipeRating — рейтинги рецептов
- Subscription — подписки пользователей

2. Проектирование микросервисной архитектуры

2.1 Выделение микросервисов

На основе анализа предметной области выделены три микросервиса:

Микросервис	Порт	Ответственность
API Gateway	3000	Аутентификация (JWT), маршрутизация, композиция (feed)
User Service	3001	Пользователи, подписки, аутентификация
Recipe Service	3002	Рецепты, комментарии, шаги, рейтинги, справочники

2.2 Database-per-service

Каждый микросервис имеет собственную базу данных PostgreSQL:

user-db (5433) — таблицы:

- `users` — пользователи
- `subscriptions` — подписки

recipe-db (5434) — таблицы:

- `recipes` — рецепты
- `recipe\_media` — медиафайлы рецептов
- `recipe\_steps` — шаги приготовления
- `recipe\_step\_media` — медиафайлы шагов
- `dish\_types` — типы блюд
- `recipe\_dish\_types` — связь рецептов с типами блюд
- `ingredients` — ингредиенты

- `recipe\_ingredients` — связь рецептов с ингредиентами
- `comments` — комментарии
- `comment\_likes` — лайки комментариев
- `saved\_recipes` — сохранённые рецепты
- `recipe\_ratings` — рейтинги рецептов

3. Межсервисное взаимодействие

3.1 HTTP (синхронное взаимодействие)

Recipe Service → User Service (внутренние HTTP-запросы):

Recipe Service не имеет своей таблицы `users`, поэтому для получения данных о пользователях (авторах рецептов, комментаторов) он обращается к User Service через внутренние эндпоинты.

Метод	Эндпоинт	Описание	Ответ
GET	`/internal/users/:userId`	Получить пользователя по ID	`{ id, username, firstName, lastName, about, role, createdAt, updatedAt }`
GET	`/internal/users/:userId/is-admin`	Проверить роль администратора	`{ isAdmin: boolean }`
GET	`/internal/users/batch?ids=1,2,3`	Получить нескольких пользователей	`[{ id, username, ...}]`
GET	`/internal/users/:userId/subscription-ids`	Получить ID подписок (для feed)	`{ authorIds: [1, 2, 3] }`

Recipe Service (внутренние эндпоинты для feed):

Метод	Эндпоинт	Описание	Ответ
GET	<code>/internal/recipes/by-authors?authorIds=1,2,3&page=1&limit=20</code>	Получить рецепты по списку авторов	<code>{ id, title, ... }</code>

3.2 RabbitMQ (асинхронное взаимодействие)

User Service → RabbitMQ → Recipe Service

User Service публикует события в RabbitMQ (exchange: `'user-exchange'`, routing key: `'user.*'`):

Событие	Когда отправляется	Данные
<code>'user.created'</code>	При регистрации нового пользователя	<code>{ userId, username }</code>
<code>'user.deleted'</code>	При удалении пользователя	<code>{ userId }</code>

Recipe Service потребляет эти события:

- При получении `'user.deleted'` — каскадно удаляет все рецепты, комментарии, лайки, рейтинги и сохранения этого пользователя.

3.3 API Gateway (композиция)

Feed (GET `/users/me/feed`):

1. API Gateway получает `'currentUserId'` из JWT-токена (через `'authMiddleware'`)

2. Делает запрос в User Service: `'GET /internal/users/{currentUserId}/subscription-ids'`

3. Получает список ID авторов, на которых подписан пользователь

4. Делает запрос в Recipe Service: `'GET /internal/recipes/by-authors?authorIds=...'`

5. Возвращает объединённый результат клиенту

4. Описание эндпоинтов микросервисов

4.1 User Service (:3001)

Публичные эндпоинты (через API Gateway):

Метод	Путь	Auth	Описание
POST	`/auth/register`	Нет	Регистрация
POST	`/auth/login`	Нет	Вход
PATCH	`/users/me/password`	JWT	Смена пароля
GET	`/users`	JWT + Admin	Список пользователей
GET	`/users/me`	JWT	Текущий пользователь
PATCH	`/users/me`	JWT	Обновить профиль
DELETE	`/users/me`	JWT	Удалить аккаунт
GET	`/users/:userId`	JWT	Пользователь по ID
DELETE	`/users/:userId`	JWT + Admin	Удалить пользователя
PATCH	`/users/:userId/role`	JWT + Admin	Изменить роль
GET	`/users/me/subscriptions`	JWT	Мои подписки
GET	`/users/me/subscribers`	JWT	Мои подписчики
GET	`/users/:userId/subscribe`	JWT	Проверить подписку
POST	`/users/:userId/subscribe`	JWT	Подписаться
DELETE	`/users/:userId/subscribe`	JWT	Отписаться
GET	`/users/:userId/subscriptions`	JWT	Подписки пользователя

GET	`/users/:userId/subscribers`	JWT	Подписчики пользователя
-----	------------------------------	-----	-------------------------

Внутренние эндпоинты (только для сервисов):

Метод	Путь	Описание
GET	`/internal/users/:userId`	Получить пользователя
GET	`/internal/users/:userId/is-admin`	Проверить админа
GET	`/internal/users/batch?ids=1,2,3`	Получить нескольких
GET	`/internal/users/:userId/subscription-ids`	ID подписок

4.2 Recipe Service (`:3002`)

Публичные эндпоинты (через API Gateway):

Метод	Путь	Auth	Описание
GET	`/recipes`	JWT	Список рецептов
POST	`/recipes`	JWT	Создать рецепт
GET	`/recipes/:recipeId`	JWT	Получить рецепт
PATCH	`/recipes/:recipeId`	JWT + Author	Обновить рецепт
DELETE	`/recipes/:recipeId`	JWT + Author/Admin	Удалить рецепт
GET	`/users/me/recipes`	JWT	Мои рецепты
GET	`/users/:userId/recipes`	JWT	Рецепты пользователя
GET	`/users/me/saved-recipes`	JWT	Мои сохранённые

GET	`/users/:userId/saved-recipes`	JWT	Сохранённые пользователя
GET	`/recipes/:recipeId/rating`	JWT	Получить рейтинг
PUT	`/recipes/:recipeId/rating`	JWT	Поставить оценку
DELETE	`/recipes/:recipeId/rating`	JWT	Удалить оценку
GET	`/recipes/:recipeId/save`	JWT	Проверить сохранение
POST	`/recipes/:recipeId/save`	JWT	Сохранить рецепт
DELETE	`/recipes/:recipeId/save`	JWT	Удалить из сохранённых
GET	`/recipes/:recipeId/comments`	JWT	Список комментариев
POST	`/recipes/:recipeId/comments`	JWT	Создать комментарий
GET	`/recipes/:recipeId/comments/:commentId`	JWT	Получить комментарий
PATCH	`/recipes/:recipeId/comments/:commentId`	JWT + Author	Обновить
DELETE	`/recipes/:recipeId/comments/:commentId`	JWT + Author/Admin	Удалить
GET	`/recipes/:recipeId/comments/:commentId/like`	JWT	Проверить лайк
POST	`/recipes/:recipeId/comments/:commentId/like`	JWT	Поставить лайк
DELETE	`/recipes/:recipeId/comments/:commentId/like`	JWT	Убрать лайк
GET	`/recipes/:recipeId/steps`	JWT	Список шагов

POST	`/recipes/:recipeId/steps`	JWT + Author	Создать шаг
GET	`/recipes/:recipeId/steps/:stepId` ,	JWT	Получить шаг
PATCH	`/recipes/:recipeId/steps/:stepId` ,	JWT + Author	Обновить шаг
DELETE	`/recipes/:recipeId/steps/:stepId` ,	JWT + Author	Удалить шаг
GET	`/dish-types`	JWT	Типы блюд
POST	`/dish-types`	JWT + Admin	Создать тип
GET	`/dish-types/:dishTypeId`	JWT	Получить тип
PATCH	`/dish-types/:dishTypeId`	JWT + Admin	Обновить тип
DELETE	`/dish-types/:dishTypeId`	JWT + Admin	Удалить тип
GET	`/ingredients`	JWT	Ингредиенты
POST	`/ingredients`	JWT + Admin	Создать ингредиент
GET	`/ingredients/:ingredientId`	JWT	Получить ингредиент
PATCH	`/ingredients/:ingredientId`	JWT + Admin	Обновить
DELETE	`/ingredients/:ingredientId`	JWT + Admin	Удалить

Внутренние эндпоинты:

Метод	Путь	Описание
GET	`/internal/recipes/by-authors?authorIds=...`	Рецепты по авторам (для feed)

4.3 API Gateway (:3000)

Метод	Путь	Описание
GET	<code>`/api/users/me/feed`</code>	Композиция: лента подписок

Все остальные запросы проксируются к соответствующим сервисам.

Возможные ошибки:

Код	Описание
400	Неверные данные (ошибка валидации Zod)
401	JWT-токен не предоставлен или недействителен
403	Недостаточно прав (не автор, не админ)
404	Рецепт не найден
502	Ошибка межсервисного взаимодействия

6. Аутентификация и безопасность

1. API Gateway проверяет JWT-токен во всех защищённых маршрутах
2. После верификации токена Gateway добавляет заголовки:
 - ``x-user-id`` — ID пользователя
 - ``x-user-role`` — роль пользователя
3. Микросервисы доверяют этим заголовкам (внутренняя сеть Docker)
4. Внутренние эндпоинты (``/internal/*``) не имеют JWT-аутентификации, так как доступны только внутри Docker-сети

Вывод

В результате проектирования была разработана микросервисная архитектура для платформы обмена рецептами, состоящая из трёх сервисов: API Gateway, User Service и Recipe Service. Каждый сервис имеет собственную базу данных (database-per-service). Межсервисное взаимодействие реализовано двумя способами: синхронное HTTP-взаимодействие (Recipe Service запрашивает данные пользователей у User Service) и асинхронное через RabbitMQ (события создания/удаления пользователя). API Gateway выполняет роль единой точки входа, обеспечивая JWT-аутентификацию, маршрутизацию и композицию данных для ленты подписок (feed). Все эндпоинты описаны с указанием методов, путей и middleware.